Учреждение «Университет «Туран»
Высшая школа информационных технологий
Образовательная программа 7M06101 — Информационные системы

ТЕХНИЧЕСКОЕ ОПИСАНИЕ

BioAuth Optimizer — программное средство многокритериальной оптимизации параметров систем биометрической аутентификации

Версия 1.0.0

Дисциплина «Методы и инструменты оптимизации»
Самостоятельная работа обучающегося № 2 (СРО2)

Выполнил:
магистрант 2 курса, группа ИС_НПМ_25р
Мекенбаев Даниэль Кайратханович

Проверил(а):
_____

Алматы, 2026

АННОТАЦИЯ

Документ описывает программное средство BioAuth Optimizer версии 1.0.0. Программа реализует методы, предложенные в статье автора «Многокритериальная оптимизация параметров систем биометрической аутентификации» (СРО1): выбор порога решения по минимуму байесовской стоимости ошибок и выбор состава факторов аутентификации методом целочисленного программирования.

Приведены назначение и область применения, математическая модель, алгоритмы и обоснование их точности, архитектура, структура данных и программный интерфейс, порядок развёртывания, методика и результаты тестирования. Отдельный раздел сопоставляет формулы статьи с модулями кода и тестами, которые их проверяют.

Таблица 1 — Основные сведения о программе

Характеристика	Значение
Наименование	BioAuth Optimizer
Версия	1.0.0 (версия вычислительного ядра 1.0.0)
Автор	Мекенбаев Даниэль Кайратханович
Тип программы	Веб-приложение (клиент-серверное), модульный монолит
Языки программирования	TypeScript 5.9 (ядро, сервер, интерфейс), SQL (PostgreSQL), Python (генерация эталонных данных)
Объём исходного текста	около 7 000 строк прикладного кода и 1 200 строк тестов
Платформа	Node.js 22, PostgreSQL 16, Docker; работает в Linux, macOS, Windows
Интерфейс пользователя	Веб-браузер (Chrome, Edge, Firefox, Safari), русский язык
Исходный код	Закрытый репозиторий GitHub DcNi3L/bioauth-optimizer

СОДЕРЖАНИЕ

АННОТАЦИЯ.....2
ВВЕДЕНИЕ.....5
1 НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ.....7
1.1 Назначение программы.....7
1.2 Пользователи.....8
1.3 Функции программы.....8
1.4 Области применения.....9
2 ПОСТАНОВКА ЗАДАЧИ И МАТЕМАТИЧЕСКАЯ МОДЕЛЬ.....10
2.1 Задача 1. Оптимальный порог.....10
2.2 Задача 2. Выбор состава факторов.....11
2.3 Точная линеаризация.....11
3 АЛГОРИТМЫ.....13
3.1 Аналитический расчёт порога с численной проверкой.....13
3.2 Точный эмпирический перебор.....14
3.3 Решатель A — полный перебор.....14
3.4 Решатель B — целочисленное программирование по фронту Парето. .15
3.5 Анализ чувствительности к бюджету.....17
3.6 Численная точность.....17
4 АРХИТЕКТУРА.....19
4.1 Архитектурный стиль.....19
4.2 Пакеты и правила зависимостей.....19
4.3 Вычислительное ядро.....20
4.4 Серверная часть.....21
4.5 Клиентская часть.....22
4.6 Соответствие принципам SOLID.....24
4.7 Сквозные аспекты.....25
4.8 Архитектурные решения.....25
5 ДАННЫЕ И ПРОГРАММНЫЙ ИНТЕРФЕЙС.....26

5.1 Модель данных....26
5.2 REST API....28
6 РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ....30
6.1 Требования к оборудованию и программному обеспечению....30
6.2 Схема развёртывания....30
6.3 Переменные окружения....31
6.4 Команды управления....31
7 ТЕСТИРОВАНИЕ И ВЕРИФИКАЦИЯ....33
7.1 Уровни тестирования....33
7.2 Золотые тесты....33
7.3 Property-based тесты....34
7.4 Непрерывная интеграция и качество кода....34
7.5 Результаты на примере статьи CPO1....35
8 СВЯЗЬ С МЕТОДАМИ CPO1....36
9 ОГРАНИЧЕНИЯ И НАПРАВЛЕНИЯ РАЗВИТИЯ....37
9.1 Ограничения версии 1.0....37
9.2 Направления развития....37
ЗАКЛЮЧЕНИЕ....39
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ....40
ПРИЛОЖЕНИЕ А. Исходные данные и результаты примера CPO1....41
ПРИЛОЖЕНИЕ Б. Структура репозитория....43

ВВЕДЕНИЕ

Система биометрической аутентификации ошибается двумя способами: пропускает злоумышленника (ложный допуск, FAR) или отказывает законному пользователю (ложный отказ, FRR). Обе ошибки управляются одним параметром — порогом сравнения τ: повышение порога уменьшает FAR и увеличивает FRR. На практике порог часто выбирают в точке равных ошибок (EER), где FAR = FRR. Такой выбор не учитывает, что цена ошибок разная: пропуск мошенника в банке обходится во много раз дороже, чем повторная попытка клиента.

Вторая инженерная задача — какие факторы включить в процедуру входа: лицо, отпечаток пальца, голос, поведенческую биометрию, аппаратный ключ FIDO2. Каждый фактор снижает вероятность ложного допуска, но увеличивает время входа, стоимость и вероятность отказа законному пользователю. Перебирать варианты вручную долго и ненадёжно.

В статье CPO1 показано, что обе задачи решаются формально: порог — минимизацией ожидаемых потерь $E(\tau) = c_{FA} \cdot FAR(\tau) + c_{FR} \cdot FRR(\tau)$, состав факторов — целочисленным программированием после логарифмирования. На модельном примере переход от EER к оптимальному порогу снижает ожидаемые потери на 41,8 %.

Цель работы — программное средство, которое применяет эти методы точно, воспроизводимо и наглядно. **Задачи:**

1) реализовать вычислительное ядро с аналитическим и эмпирическим режимами расчёта порога и двумя независимыми решателями задачи выбора факторов;
2) обеспечить доказуемую точность: совпадение с независимым расчётом в SciPy, сверка решателей между собой, контроль погрешности плавающей точки;
3) построить серверное приложение с хранением экспериментов и веб-интерфейс с графиками;

4) обеспечить запуск одной командой в Docker и автоматическую проверку в CI;
5) подготовить документацию и воспроизвести на программе числовой пример статьи CPO1.

1 НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

1.1 Назначение программы

BioAuth Optimizer — инструмент поддержки решений для проектировщика системы биометрической аутентификации. Программа отвечает на вопрос «как настроить систему аутентификации», а не «пропустить ли этого человека»: сама она пользователей не аутентифицирует.

Программа решает три задачи:

- **оптимальный порог** — по параметрам распределений оценок сходства или по файлу реальных оценок (CSV) и заданным стоимостям ошибок находит порог τ* с минимальными ожидаемыми потерями и сравнивает его с порогом EER;
- **оптимальный состав факторов** — по каталогу факторов (FAR, FRR, задержка, стоимость, наличие обнаружения атак на предъявление — PAD) и ограничениям находит набор с минимальными потерями и доказывает его оптимальность;
- **чувствительность к бюджету** — показывает, при каком бюджете меняется оптимальный набор и во сколько раз растут потери при ужесточении бюджета.

Каждый расчёт сохраняется как неизменяемый эксперимент: входные данные, результат, версия ядра и время расчёта. Результаты экспортируются в JSON, CSV и PNG.

1.2 Пользователи

Таблица 2 — Пользователи программы и решаемые ими задачи

Роль	Решение, которое помогает принять
Архитектор, инженер информационной безопасности	Какой порог выставить в сравнителе; какие факторы включить в политику входа при ограничениях на задержку и бюджет
Владелец продукта (банк, госуслуги, вуз)	Сколько стоит ужесточение требований; где выгоднее потратить бюджет
Интегратор, специалист по закупкам	Какой алгоритм сравнения выбрать: сравнение ожидаемых потерь по оценкам пилотного проекта
Аудитор, служба комплаенс	Обоснован ли выбранный порог: неизменяемый эксперимент с входом, результатом и версией ядра
Исследователь, магистрант	Как метод оптимизации меняет качество системы; воспроизводимые расчёты для статей

1.3 Функции программы

Таблица 3 — Функциональные требования

Код	Функция
FR-T1	Ввод параметров нормальных распределений оценок μ_G, σ_G, μ_I, σ_I и стоимостей ошибок c_{FA}, $c_{FR} > 0$
FR-T2	Загрузка CSV с оценками (колонки `score`, `label`; label — genuine или impostor), до 1 000 000 строк
FR-T3	Расчёт точки равных ошибок: τ_{EER}, FAR = FRR, $E(\tau_{EER})$
FR-T4	Расчёт оптимального порога $\tau^* = \arg\min E(\tau)$ и значений FAR, FRR, E в нём
FR-T5	Относительное снижение потерь $\Delta = 1 - E(\tau^*)/E(\tau_{EER})$
FR-T6	Необязательное ограничение $FAR(\tau) \le FAR_{max}$
FR-T7	Графики $FAR(\tau)$, $FRR(\tau)$, $E(\tau)$ на логарифмической шкале и ROC-кривая
FR-F1	Каталог факторов: код, название, FAR, FRR, задержка, стоимость, признак PAD; от 1 до 50 факторов
FR-F2	Ограничения: максимальная задержка, бюджет, FAR_{max}, минимальное число факторов с PAD
FR-F3	Правило объединения «И»: $FAR = \prod FAR_i$, $FRR = 1 - \prod(1 - FRR_i)$; задержка и стоимость суммируются
FR-F4	Выбор решателя: полный перебор, целочисленное программирование или автоматически
FR-F5	Результат: оптимальный набор, метрики, рейтинг допустимых наборов, активные ограничения
FR-F6	Анализ чувствительности к бюджету: точки смены оптимума и рост потерь
FR-F7	Диагностика недопустимой задачи: какие ограничения отсекают наборы

Код	Функция
FR-G1…G5	Пример статьи CPO1 одной кнопкой; сохранение и история экспериментов; экспорт; управление каталогами

1.4 Области применения

Метод применим везде, где решение принимается по оценке сходства биометрического образца с эталоном и цена ошибок известна хотя бы по порядку величины (таблица 4).

Таблица 4 — Примеры применения

Отрасль	Задача	Пример постановки
Банки и МФО	Порог сравнения лица при удалённой идентификации; факторы для входа и подтверждения операций	Мошеннический кредит в сотни раз дороже отказа клиенту: $c_{FA}/c_{FR} = 200$; набор при задержке ≤ 3 с
Госуслуги	Вход по биометрии на массовой аудитории	Ложный отказ ведёт к визиту в ЦОН — отношение c_{FA}/c_{FR} ниже, чем в банке
Образование	Прокторинг экзаменов, вход в LMS, учёт посещаемости	c_{FA} — сдача экзамена за другого, c_{FR} — сорванный экзамен
Контроль доступа	Турникеты, серверные, лаборатории	Турникет: задержка ≤ 1 с; серверная: $FAR_{max} = 10^{-7}$
Корпоративный доступ	Разный порог для разных операций (step-up)	Просмотр почты $c_{FA} = 5$, перевод средств $c_{FA} = 1000$ — два порога для одного сравнителя
Вендоры, интеграторы	Сравнение алгоритмов на пилоте	Оценки двух алгоритмов на одной выборке: у кого меньше $E(\tau^*)$ при c_{FA}/c_{FR} заказчика

2 ПОСТАНОВКА ЗАДАЧИ И МАТЕМАТИЧЕСКАЯ МОДЕЛЬ

Обозначения соответствуют статье CPO1: s — оценка сходства, правило допуска $s \geq \tau$; G — законные пользователи (genuine), I — злоумышленники (impostor).

2.1 Задача 1. Оптимальный порог

Ожидаемые потери системы при пороге τ — формула (1) статьи:

$$E(\tau) = c_{FA} \cdot FAR(\tau) + c_{FR} \cdot FRR(\tau) \rightarrow min, \quad c_{FA} > 0, \; c_{FR} > 0 \tag{1}$$

$$FAR(\tau) = P(s_I \geq \tau), \quad FRR(\tau) = P(s_G < \tau) \tag{2}$$

FAR(τ) монотонно убывает, FRR(τ) монотонно возрастает, поэтому задача сводится к поиску компромисса, который задают стоимости c_{FA} и c_{FR}.

Параметрический режим

Оценки каждой группы распределены нормально: $s_G \sim N(\mu_G, \sigma_G)$, $s_I \sim N(\mu_I, \sigma_I)$, $\mu_G > \mu_I$, σ — стандартное отклонение. Вероятности ошибок вычисляются через дополнительную функцию ошибок erfc, а не как 1 − Ф, чтобы не терять точность в хвостах распределений (FAR порядка 10^{-6}…10^{-15}):

$$FAR(\tau) = \tfrac{1}{2} \cdot erfc((\tau - \mu_I) / (\sigma_I\sqrt{2})) \tag{3}$$

$$FRR(\tau) = \tfrac{1}{2} \cdot erfc((\mu_G - \tau) / (\sigma_G\sqrt{2})) \tag{4}$$

Точка равных ошибок получается в замкнутом виде из условия $(\tau - \mu_I)/\sigma_I = (\mu_G - \tau)/\sigma_G$:

$$\tau_{EER} = (\mu_I\sigma_G + \mu_G\sigma_I) / (\sigma_G + \sigma_I) \tag{5}$$

$$EER = \tfrac{1}{2} \cdot erfc(d / \sqrt{2}), \quad d = (\mu_G - \mu_I) / (\sigma_G + \sigma_I) \tag{6}$$

Условие стационарности $E'(\tau) = -c_{FA}f_I(\tau) + c_{FR}f_G(\tau) = 0$, где f — плотности нормальных распределений, после логарифмирования становится квадратным уравнением:

$$Q(\tau) = a\tau^2 + b\tau + c = 0 \tag{7}$$

$$a = 1/(2\sigma_G^2) - 1/(2\sigma_I^2), \quad b = \mu_I/\sigma_I^2 - \mu_G/\sigma_G^2$$

$$c = \mu_G^2/(2\sigma_G^2) - \mu_I^2/(2\sigma_I^2) + ln(c_{FA}/c_{FR}) + ln(\sigma_G/\sigma_I)$$

Знак E′(τ) совпадает со знаком −Q(τ). Поэтому корень уравнения — локальный минимум E, если Q′(τ) = 2aτ + b < 0. Эта проверка точна и не требует второй производной, которая в хвостах обращается в машинный ноль.

Эмпирический режим

Для выборок оценок G объёмом n_G и I объёмом n_I ошибки — доли выборки:

$$FAR(\tau) = \#\{s \in I : s \geq \tau\} / n_I, \quad FRR(\tau) = \#\{s \in G : s < \tau\} / n_G \tag{8}$$

Функция E(τ) кусочно-постоянна и меняется только в значениях оценок, поэтому точный минимум находится перебором всех различных значений, а не численной оптимизацией.

2.2 Задача 2. Выбор состава факторов

Факторы i = 1…n описываются вероятностями FAR_i, FRR_i, задержкой l_i, стоимостью c_i и признаком PAD. Переменная $x_i \in \{0, 1\}$ показывает, включён ли фактор. При правиле «И» злоумышленник должен пройти все включённые факторы — формулы (2)–(3) статьи:

$$FAR(x) = \prod FAR_i^{x\sim i\sim}, \quad FRR(x) = 1 - \prod (1 - FRR_i)^{x\sim i\sim} \tag{9}$$

$$L(x) = \Sigma\, l_i x_i, \quad C(x) = \Sigma\, c_i x_i, \quad P(x) = \Sigma_{i \in PAD}\, x_i \tag{10}$$

$$min\ E(x) = c_{FA} \cdot FAR(x) + c_{FR} \cdot FRR(x) \tag{11}$$

$$L(x) \leq L_{max}, \ C(x) \leq B, \ P(x) \geq P_{min}, \ FAR(x) \leq FAR_{max}, \ \Sigma\, x_i \geq 1 \tag{12}$$

2.3 Точная линеаризация

В статье CPO1 предложено логарифмировать задачу, чтобы она стала линейной по x. В программе использована точная форма этой идеи. Обе ошибки набора — монотонные функции двух линейных форм:

$$L(x) = \Sigma\, \ln(FAR_i) \cdot x_i, \quad FAR(x) = e^{L(x)} \tag{13}$$

$$\Lambda(x) = -\Sigma\, \ln(1 - FRR_i) \cdot x_i \geq 0, \quad FRR(x) = 1 - e^{-\Lambda(x)} \tag{14}$$

В статье второе слагаемое приближалось суммой Σ $FRR_i x_i$. Форма Λ совпадает с ней в первом порядке, но выражает FRR точно, поэтому

погрешности линеаризации нет. Ограничение $FAR(x) \le FAR_{max}$ также становится линейным: $L(x) \le \ln FAR_{max}$.

Целевая функция $E(x) = c_{FA} \cdot e^{L(x)} + c_{FR} \cdot (1 - e^{-\Lambda(x)})$ возрастает и по L, и по Λ. Если набор y не лучше x по обеим формам, то $E(y) \ge E(x)$. Следовательно, оптимум лежит на **фронте Парето** по паре (Λ, L), и для точного решения достаточно обойти этот фронт (раздел 3.4).

3 АЛГОРИТМЫ

3.1 Аналитический расчёт порога с численной проверкой

Алгоритм `optimizeGaussianThreshold` (модуль `gaussian-analytic.optimizer.ts`):

1) Кандидаты: вещественные корни уравнения (7), в которых E имеет локальный минимум, и два предельных случая — «допускать всех» ($E \to c_{FA}$) и «отвергать всех» ($E \to c_{FR}$). Корни вычисляются устойчивой формулой $q = -\tfrac{1}{2}(b + \mathrm{sign}(b)\cdot\sqrt{D})$, $\tau_1 = q/a$, $\tau_2 = c/q$; при $|a| < 10^{-12}$ ($\sigma_G \approx \sigma_I$) уравнение линейное, $\tau = -c/b$.
2) Если задано FAR_{max}, вычисляется $\tau_{min} = \mu_I + \sigma_I\sqrt{2}\cdot\mathrm{erfcinv}(2\cdot FAR_{max})$; корни левее τ_{min} отбрасываются, τ_{min} добавляется в кандидаты.
3) τ^* — кандидат с минимальным E. Если побеждает предельный случай, результат помечается как вырожденный, и интерфейс объясняет, что при таких стоимостях порог не имеет смысла. Корень, выигрывающий у предельного случая меньше чем 10^{-9} относительно, тоже считается вырожденным: он лежит в дальнем хвосте, где E плоская до машинной точности.
4) Независимая проверка: сетка из 10^4 точек на $[\mu_I - 10\sigma_I, \mu_G + 10\sigma_G]$ находит грубый минимум, бисекция по производной $E'(\tau)$, вычисленной напрямую через плотности, уточняет его до машинной точности. При $|\tau_{аналит} - \tau_{числ}| > 10^{-7}$ результат помечается как расхождение и записывается в журнал как ошибка.
5) Вычисляется $\Delta = 1 - E(\tau^*)/E(\tau_{EER})$.

Золотое сечение для проверки не подходит: $E(\tau)$ у минимума плоская, и сравнение значений функции даёт точность лишь около 10^{-8}. Бисекция по знаку производной сходится до машинной точности. На примере статьи расхождение аналитического и численного решений — $2{,}2\cdot10^{-16}$.

3.2 Точный эмпирический перебор

Алгоритм `optimizeEmpiricalThreshold` (модуль `empirical-sweep.optimizer.ts`), сложность $O((n_G + n_I)\cdot\log(n_G + n_I))$:

1) Выборки G и I сортируются; T — упорядоченные уникальные значения $G \cup I$ и $+\infty$ («отвергать всех»).
2) Два указателя проходят T и поддерживают счётчики, из которых для каждого t_k получаются FAR_k, FRR_k, E_k.
3) τ^* — значение с минимальным E_k. Решения одинаковы на всём интервале $(t_{k-1}, t_k]$, поэтому возвращаются интервал и его середина как рекомендуемый порог — она устойчивее к новым данным.
4) При равенстве E выбирается меньший FAR (приоритет безопасности), затем меньший τ.
5) EER — точка, минимизирующая $|FAR_k - FRR_k|$.
6) Если злоумышленников в выборке меньше $3/FAR_{max}$, выдаётся предупреждение: данных недостаточно, чтобы статистически подтвердить такой FAR (правило трёх).

3.3 Решатель A — полный перебор

Эталон точности для $n \le 20$. Перебираются все $2^n - 1$ непустых подмножеств, E считается по точной нелинейной формуле (11) в логарифмическом пространстве: $\log FAR = \Sigma x_i \cdot \ln FAR_i$, $\log Surv = \Sigma x_i \cdot \mathrm{log1p}(-FRR_i)$, $FRR = -\mathrm{expm1}(\log Surv)$. Функции log1p и expm1 сохраняют точность при малых FRR.

Метрики каждого подмножества вычисляются заново суммированием в фиксированном порядке индексов. Инкрементальные обновления (код Грея) отвергнуты: при миллионе прибавлений и вычитаний накапливается погрешность, сравнимая с допуском ε, и результат зависел бы от порядка обхода. Для $n = 20$ полный перебор занимает меньше секунды. Хранятся лучшие K наборов (по умолчанию 100) и счётчики отсечений по каждому

ограничению. Порядок при равенстве E детерминирован: E, FAR, C, L, затем коды факторов.

3.4 Решатель B — целочисленное программирование по фронту Парето

Это метод CPO1: логарифмирование делает задачу линейной по x и сводит её к целочисленному программированию. В статье назван решатель CP-SAT; в Node.js в той же роли используется MILP-решатель HiGHS (WebAssembly-сборка, пакет `highs`). Решатель B работает до n = 50 и не зависит от HiGHS напрямую: он обращается к интерфейсу `MilpPort`, реализацию которого подставляет адаптер `HighsMilpAdapter`.

Алгоритм обходит фронт Парето методом ε-ограничений с отсечениями «no-good» (рисунок 1):

```
best ← ∞;  L_bound ← +∞;  𝒞 ← ∅
repeat
    x̂  ← argmin Λ(x)  при (12), L(x) ≤ L_bound, вне 𝒞        // MILP 1
    если x̂ не существует → стоп: фронт пройден, best — оптимум
    LB ← cFR·(1 − e^{−(Λ(x̂) − δ)})
    если LB ≥ best → стоп: оптимум доказан
    x̂′ ← argmin L(x)  при тех же ограничениях и Λ(x) ≤ Λ(x̂) + δ  // MILP 2
    для x ∈ {x̂, x̂′}: точное E(x); если допустим — best ← min(best, E(x));
                      𝒞 ← 𝒞 ∪ {x}
    если x̂′ допустим по точной проверке: L_bound ← L(x̂′)
until итераций > 2ⁿ
```

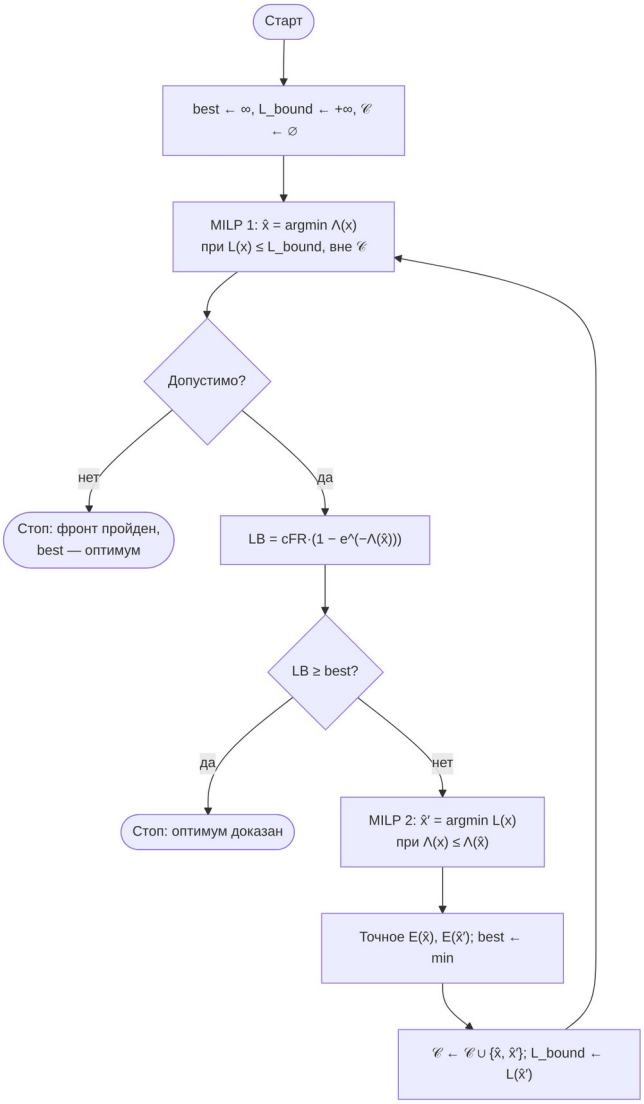


Рисунок 1 — Алгоритм целочисленного программирования с сертификатом оптимальности

Почему результат точный. Наборы с $L > L_{bound}$ доминируются найденной точкой $\hat{x}'$: у них Λ не меньше ($\hat{x}$ минимизировал Λ), а L больше, поэтому их исключение не теряет оптимум. Каждый следующий шаг MILP 1 решается на подмножестве предыдущего, поэтому $\Lambda(\hat{x})$ не убывает, и $LB = c_{FR}\cdot(1 - e^{-\Lambda(\hat{x})})$ ограничивает снизу E всех ещё не просмотренных наборов. Отсечения $\mathscr{C}$ гарантируют, что ни один набор не просматривается дважды, поэтому алгоритм конечен. Запас $\delta = 10^{-9}$ на допуски MILP направлен в безопасную сторону: он занижает LB и расширяет допустимое множество. Окончательная допустимость каждого набора проверяется точной формулой, а не по ответу MILP.

Трассировка на примере статьи. Итерация 1: $\hat{x} = \hat{x}' = F2+F5$ ($\Lambda = 0{,}020126$), best = 0,019925, $L_{bound} = \ln(5\cdot10^{-11})$. Итерация 2: при $FAR \le 5\cdot10^{-11}$ остаётся F2+F4+F5 с $\Lambda = 0{,}071419$, $LB = 0{,}068929 \ge 0{,}019925$ — остановка. Две итерации, оптимальность доказана; сертификат (число итераций, нижняя граница, лучшее значение) возвращается вместе с результатом и показывается в интерфейсе.

История разработки. Первая версия минимизировала $\Sigma\ FRR_i x_i$ и останавливалась по оценке $c_{FR}\cdot(1 - e^{-S})$. Тесты на случайных задачах выявили два плохих случая: если у факторов FRR = 0, оценка равна нулю и решатель перебирал все наборы; при множестве «ничьих» он обходил их по одному. Обход фронта Парето устранил оба случая; каждый закреплён регрессионным тестом.

Автоматический режим: при $n \le 20$ используется полный перебор (даёт полный рейтинг), иначе — целочисленное программирование.

3.5 Анализ чувствительности к бюджету

Для диапазона бюджета $[B_{от}, B_{до}]$ с шагом h наборы, допустимые без учёта бюджета, сортируются по E один раз; для каждого B берётся первый набор с $C \le B$. Возвращаются точки (B, E*, набор) и точки смены оптимума с отношением E_{new}/E_{old} — «цена» ужесточения бюджета из статьи. На примере статьи: $B \in [0{,}9;\ 1{,}6)$ — F2+F4; $[1{,}6;\ 2{,}1)$ — F1+F2; $B \ge 2{,}1$ — F2+F5. Сокращение бюджета с 2,5 до 2,0 увеличивает потери в 0,034700/0,019925 = 1,74 раза.

3.6 Численная точность

Все нестрогие ограничения проверяются с допуском: $lhs \le rhs + \varepsilon\cdot\max(1, |rhs|)$, $\varepsilon = 10^{-9}$; ограничение по FAR — в логарифмической форме. Без допуска набор F2+F4 с $FAR = 5\cdot10^{-5}\cdot2\cdot10^{-2} = 1{,}0000000000000002\cdot10^{-6}$ (так результат представлен в IEEE 754) ошибочно отбрасывается при $FAR_{max} = 10^{-6}$, и вместо 6 допустимых наборов получается 5. Этот случай закреплён регрессионным

тестом. Специальные функции erfc, erfcinv, log1p, expm1 берутся из библиотеки @stdlib с проверенной точностью.

4 АРХИТЕКТУРА

4.1 Архитектурный стиль

Программа построена как **модульный монолит в монорепозитории**. Три развёртываемых контейнера: `web` (интерфейс на Next.js), `api` (сервер на NestJS) и `db` (PostgreSQL) — рисунок 2. Вся математика сосредоточена в пакете `@bioauth/optimization-core` — чистом TypeScript без зависимостей от фреймворков, базы данных и HTTP. Сервер — тонкая оболочка над ядром (валидация, сохранение, HTTP), интерфейс только отображает результаты и ничего не вычисляет. Так у результатов расчёта один источник правды.

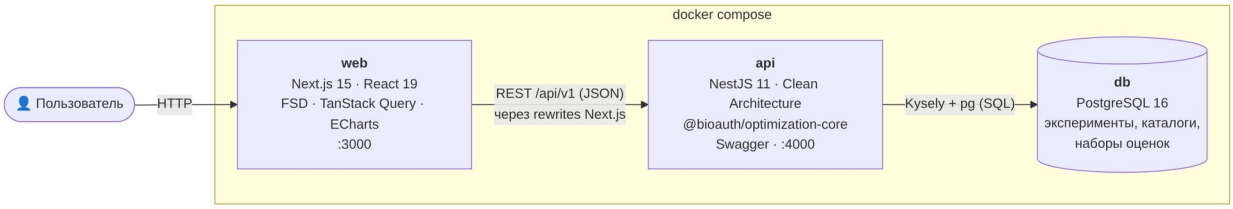


Рисунок 2 — Контейнеры системы (модель C4, уровень 2)

Очереди (RabbitMQ) и кэш (Redis) осознанно не используются: самый долгий расчёт (целочисленное программирование для 50 факторов) укладывается в секунды и выполняется синхронно. Точка расширения для длительных задач предусмотрена в архитектуре.

4.2 Пакеты и правила зависимостей

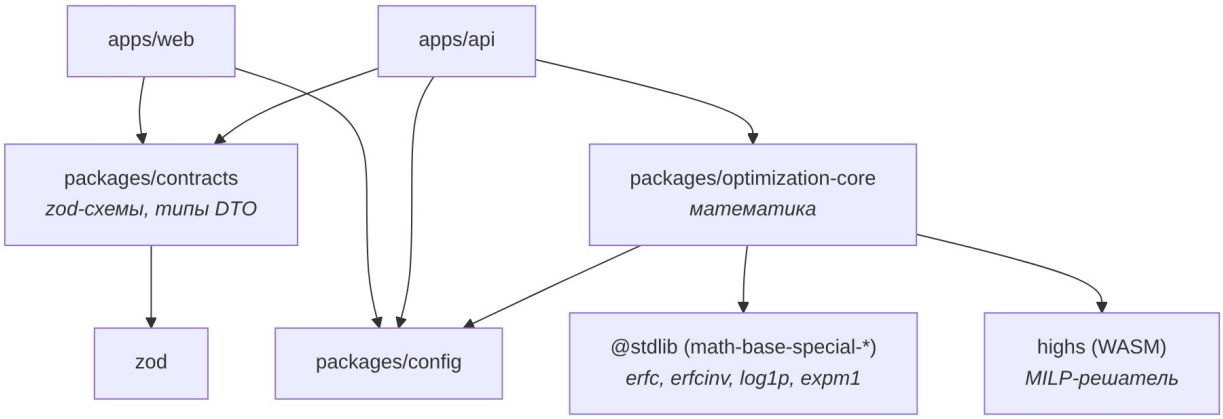


Рисунок 3 — Пакеты монорепозитория и направления зависимостей

Таблица 5 — Пакеты монорепозитория

Пакет	Назначение	Зависимости
`packages/optimization-core`	Вычислительное ядро: порог, выбор факторов, решатели, чувствительность	@stdlib (специальные функции), highs (MILP)
`packages/contracts`	zod-схемы входных и выходных данных API, общие для сервера и интерфейса	zod
`apps/api`	Сервер NestJS: HTTP, валидация, хранение, Swagger	ядро, контракты, Kysely, pg, pino
`apps/web`	Интерфейс Next.js: формы, графики, история	контракты, React, TanStack Query, ECharts

Правила зависимостей проверяются линтером при каждом коммите: ядро не импортирует фреймворки, драйверы БД, файловую систему, не читает переменные окружения и не использует Math.random и Date.now (расчёты детерминированы); интерфейс не импортирует ядро.

4.3 Вычислительное ядро

Ключевые абстракции ядра показаны на рисунке 4. Модель оценок (`ScoreModel`), оптимизатор порога (`ThresholdOptimizer`), правило объединения (`FusionRule`), решатель (`FactorSelectionSolver`) и порт MILP (`MilpPort`) — интерфейсы; каждая реализация заменяема без изменения остального кода.

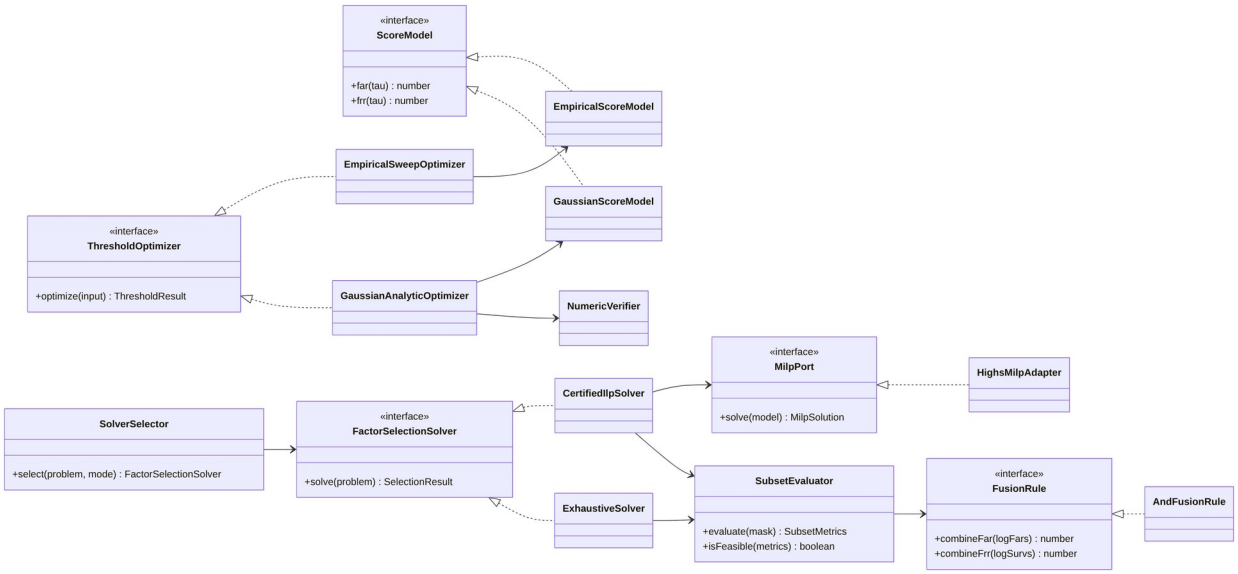


Рисунок 4 — Диаграмма классов вычислительного ядра

Таблица 6 — Модули вычислительного ядра

Модуль	Ответственность
`numeric/`	Обёртки специальных функций, допуск сравнения ε (tolerance.ts), бисекция
`threshold/gaussian-score-model.ts`, `empirical-score-model.ts`	FAR(τ), FRR(τ) для нормальной модели и для выборки
`threshold/optimizers/`	Аналитический оптимизатор (3.1) и эмпирический перебор (3.2)
`threshold/verification/numeric-verifier.ts`	Независимая численная проверка τ*
`threshold/curves.ts`	Точки кривых FAR/FRR/E и ROC для графиков
`factor-selection/subset-evaluator.ts`	Метрики набора в логарифмическом пространстве и точная проверка допустимости
`factor-selection/fusion/and-fusion.rule.ts`	Правило объединения «И»
`factor-selection/solvers/exhaustive.solver.ts`	Решатель A — полный перебор (3.3)
`factor-selection/solvers/certified-ilp.solver.ts`	Решатель B — ЦЛП по фронту Парето с сертификатом (3.4)
`factor-selection/solvers/highs-milp.adapter.ts`	Реализация порта MilpPort на HiGHS
`factor-selection/solvers/solver-selector.ts`	Выбор решателя в автоматическом режиме
`factor-selection/sensitivity/budget-sweep.ts`	Анализ чувствительности к бюджету (3.5)
`examples/article-sro1.ts`	Данные модельного примера раздела 4 статьи СРО1

4.4 Серверная часть

Сервер построен по принципам Clean (гексагональной) архитектуры (рисунок 5). Каждый модуль разделён на слои: представление (контроллеры, валидация, Swagger), приложение (сценарии использования и порты-интерфейсы) и инфраструктура (репозитории на Kysely, разбор CSV). Доменный слой — само ядро. Зависимости направлены только внутрь: сценарий использования знает интерфейс `ExperimentRepository`, но не знает Kysely; реализации подставляются через DI-токены NestJS.

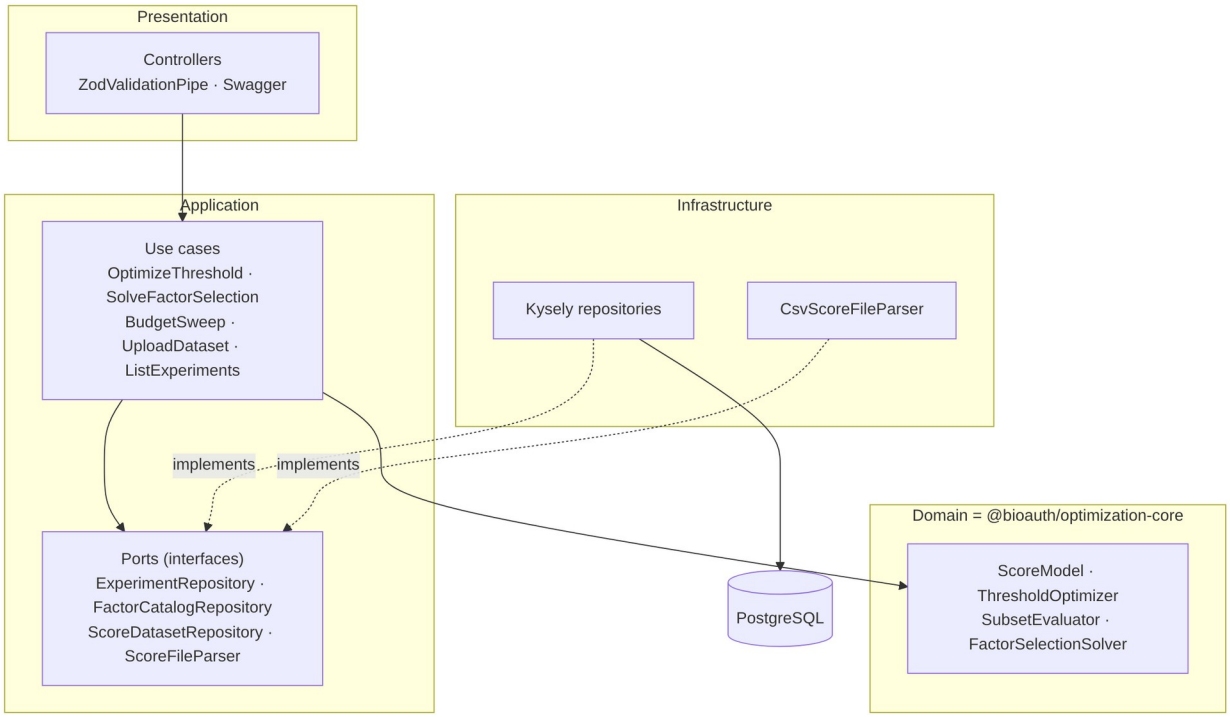


Рисунок 5 — Слои серверной части

Таблица 7 — Модули сервера

Модуль	Назначение
threshold	Расчёт оптимального порога (параметрический режим и по набору оценок)
factor-selection	Выбор состава факторов и анализ чувствительности к бюджету
factor-catalogs	Создание, чтение, изменение и удаление каталогов факторов; предустановленный каталог CPO1
score-datasets	Потоковая загрузка CSV с оценками, хранение в компактном двоичном виде, гистограмма
experiments	Сохранение и просмотр неизменяемых экспериментов, постраничная выдача
examples	Данные примера статьи CPO1
shared	Конфигурация (валидация окружения), база данных и миграции, обработка ошибок, журналирование, проверки здоровья, Swagger

4.5 Клиентская часть

Интерфейс построен на Next.js 15 (App Router) по методологии Feature-Sliced Design (рисунок 6). Слой импортирует только нижележащие слои, срезы одного слоя не импортируют друг друга, обращение к срезу — только через его публичный интерфейс `index.ts`. Правила проверяет линтер steiger. Папка

`app/` в корне содержит только маршруты Next.js, которые реэкспортируют страницы из `src/pages`.

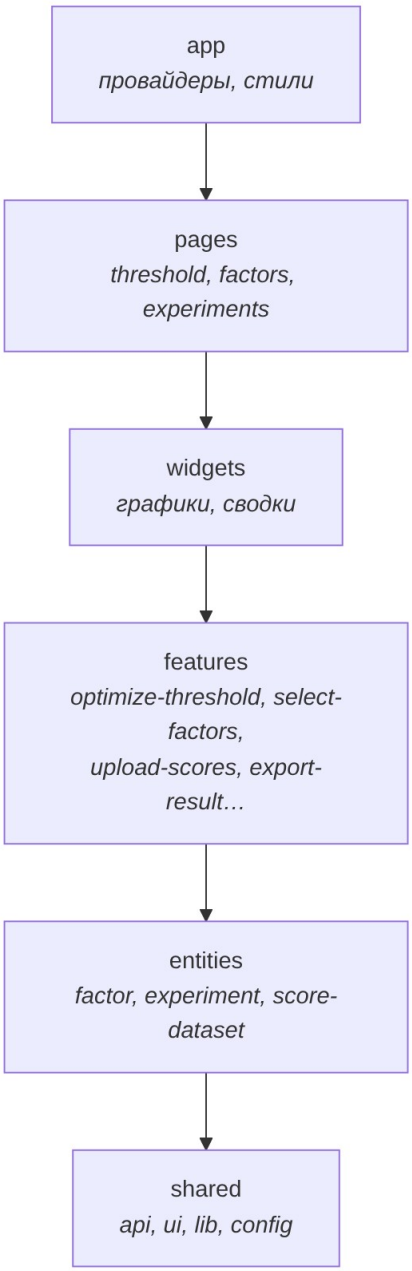


Рисунок 6 — Слои Feature-Sliced Design

Таблица 8 — Слои клиентской части

Слой	Содержимое
app	Провайдеры (кэш запросов), глобальные стили
pages	Главная, «Порог», «Состав факторов», «История», карточка эксперимента
widgets	Графики ошибок и ROC, сводки результатов, рейтинг наборов, чувствительность к бюджету, таблица экспериментов
features	Расчёт порога, загрузка CSV, выбор факторов, редактирование каталога, анализ бюджета, пример статьи, экспорт
entities	Фактор, каталог, эксперимент, набор оценок: типы, отображение, запросы к API
shared	HTTP-клиент с проверкой ответов по zod-схемам, UI-кит, форматирование чисел ($5{,}0 \cdot 10^{-11}$), конфигурация

Серверное состояние хранит TanStack Query, состояние форм — локальные модели фич; глобального хранилища нет. Формы проверяются той же zod-схемой, что и на сервере, поэтому ошибка ввода показывается рядом с полем до отправки запроса.

4.6 Соответствие принципам SOLID

Таблица 9 — Принципы SOLID в коде

Принцип	Где и как соблюдается
S — единственная ответственность	`SubsetEvaluator` только считает метрики и допустимость; решатель только перебирает; `ExperimentRecorder` только сохраняет; контроллер только переводит HTTP в команду сценария. На клиенте фича расчёта не рисует графики — это делает виджет
O — открытость/закрытость	Новый решатель, правило объединения или модель оценок добавляется новой реализацией интерфейса без правки существующего кода; селектор получает реестр решателей через DI
L — подстановка Лисков	Любой решатель возвращает `SelectionResult` с одинаковым контрактом; один и тот же набор тестов проверяет обе реализации
I — разделение интерфейсов	`ScoreModel` содержит только far и frr; кривые строит отдельный модуль. Порты репозиториев узкие: save, findById, list
D — инверсия зависимостей	Сценарии зависят от портов (`ExperimentRepository`, `MilpPort`); реализации (Kysely, HiGHS) подставляются через DI. Решатель B в тестах получает заглушку MilpPort

4.7 Сквозные аспекты

– **Валидация.** Одна zod-схема из `@bioauth/contracts` используется в форме интерфейса, в конвейере валидации NestJS и для генерации спецификации OpenAPI.
– **Ошибки** возвращаются в формате RFC 7807 (application/problem+json): некорректные данные (400), недопустимая задача с диагностикой ограничений (422), слишком большая задача (413), не найдено (404), внутреннее расхождение проверки (500).
– **Журналирование** — структурированные JSON-логи pino с идентификатором запроса, длительностью расчёта, выбранным решателем и числом итераций.
– **Воспроизводимость.** В каждый эксперимент записывается версия ядра и хеш канонического JSON входных данных.

4.8 Архитектурные решения

Каждое существенное решение оформлено записью ADR (Architecture Decision Record) в каталоге `docs/adr` (таблица 10).

Таблица 10 — Записи архитектурных решений версии 1

№	Решение
0001	Модульный монолит в монорепозитории (pnpm, Turborepo)
0002	Чистое вычислительное ядро без зависимостей от фреймворков
0003	Сервер на NestJS по принципам Clean Architecture
0004	Интерфейс на Next.js по Feature-Sliced Design
0005	PostgreSQL без Redis и RabbitMQ
0006	Аналитический расчёт порога с численной проверкой
0007	Два независимых решателя задачи выбора факторов
0008	Численная точность и допуски сравнения
0009	Общие zod-контракты для сервера и интерфейса
0010	Развёртывание в Docker Compose
0011	Kysely и SQL-миграции вместо Prisma

5 ДАННЫЕ И ПРОГРАММНЫЙ ИНТЕРФЕЙС

5.1 Модель данных

Данные хранятся в PostgreSQL 16 (рисунок 7). Принципы модели:

– эксперимент неизменяем и самодостаточен: полный вход (с копией факторов, а не ссылкой на каталог) и полный результат хранятся в jsonb, поэтому изменение каталога не меняет историю;
– инварианты закреплены в схеме (CHECK, NOT NULL, UNIQUE), а не только в коде: например, $0 < far < 1$, $0 \leq frr < 1$, длина массива оценок равна их числу × 8 байт;
– оценки хранятся компактно в bytea (Float64): миллион оценок занимает 8 МБ; повторная загрузка того же файла не дублирует данные (уникальный SHA-256);
– мягкого удаления нет: каталог удаляется физически, ссылки из экспериментов обнуляются.

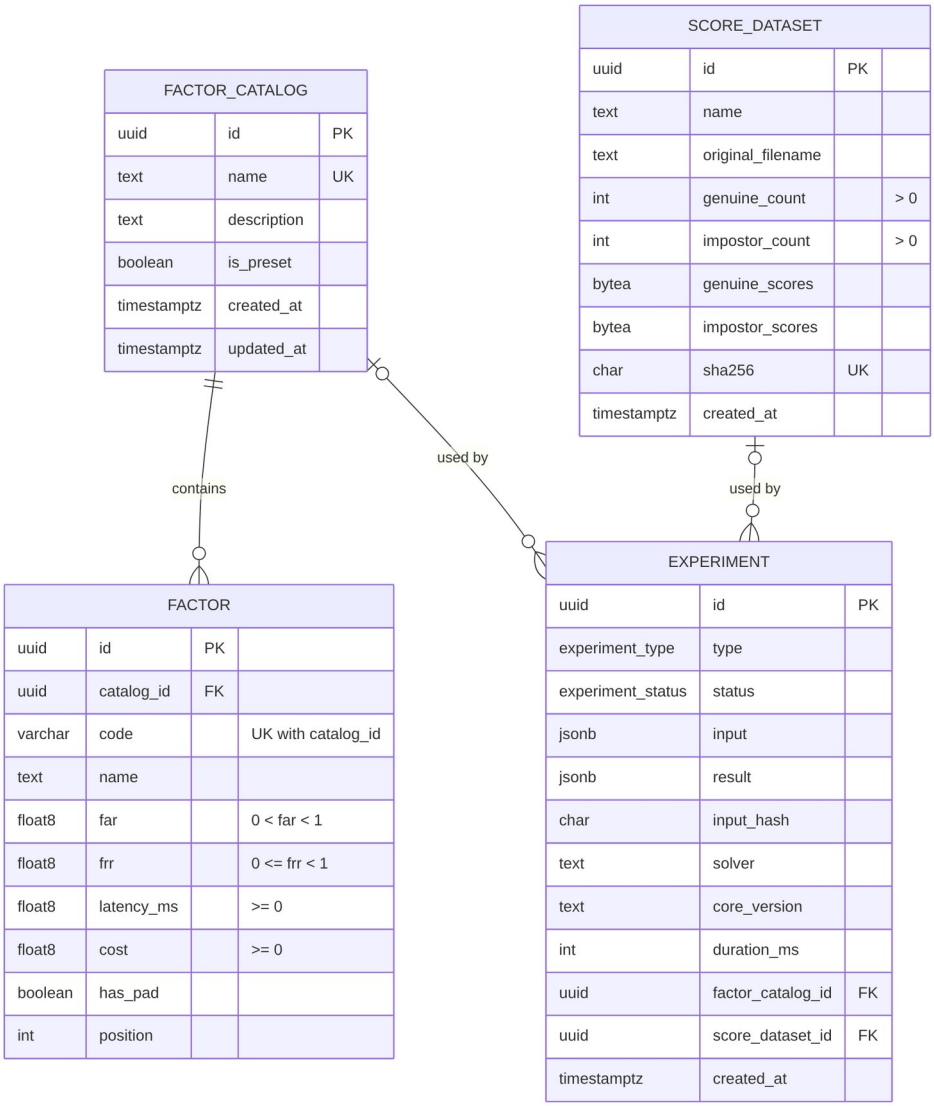


Рисунок 7 — Схема базы данных

Таблица 11 — Таблицы базы данных

Таблица	Содержимое
factor_catalog	Каталоги факторов: название (уникально), описание, признак предустановленного
factor	Факторы каталога: код (уникален в каталоге), FAR, FRR, задержка, стоимость, PAD, порядок
score_dataset	Наборы оценок: число и массивы оценок законных пользователей и злоумышленников, SHA-256 файла
experiment	Эксперименты: тип, статус, вход и результат (jsonb), хеш входа, решатель, версия ядра, длительность

Схема создаётся SQL-миграциями из `apps/api/migrations`. Мигратор применяет их при старте контейнера api под рекомендательной блокировкой

PostgreSQL (безопасно при нескольких экземплярах) и затем идемпотентно добавляет каталог примера СРО1.

5.2 REST API

Базовый путь — `/api/v1`, формат JSON. Документация Swagger доступна по адресу `/api/docs` и генерируется из тех же zod-схем, что валидируют запросы. Числа передаются без округления; форматирует их только интерфейс.

Таблица 12 — Методы API

Метод и путь	Назначение
POST /threshold/optimize	Оптимальный порог (параметрический режим или по сохранённому набору оценок)
POST /score-datasets	Загрузка CSV с оценками (multipart, поле file)
GET /score-datasets/:id	Метаданные набора: объёмы, диапазон, гистограмма
POST /factor-selection/solve	Оптимальный состав факторов
POST /factor-selection/budget-sweep	Анализ чувствительности к бюджету
GET, POST /factor-catalogs	Список и создание каталогов
GET, PUT, DELETE /factor-catalogs/:id	Чтение, замена, удаление каталога
GET /experiments	История экспериментов с фильтром по типу и постраничной выдачей
GET /experiments/:id	Эксперимент целиком: вход и результат
GET /examples/sro1	Данные примера статьи СРО1
GET /health/live, /health/ready	Проверки живости и готовности (ready проверяет БД)

Пример запроса расчёта порога для данных статьи и фрагмент ответа:

```
POST /api/v1/threshold/optimize
{ "model": { "kind": "gaussian",
             "genuine":  { "mean": 0.72, "std": 0.09 },
             "impostor": { "mean": 0.30, "std": 0.11 } },
  "costs": { "cFA": 10, "cFR": 1 }, "constraints": { "farMax": null } }

200 OK
{ "eer":     { "tau": 0.531, "far": 0.017864420562817, "cost": 0.196508626190982 },
  "optimal": { "tau": 0.581785716220834, "far": 0.00520823745159,
               "frr": 0.062304237539322, "cost": 0.114386612055219 },
  "lossReduction": 0.41790539,
  "verification": { "status": "ok" },
  "meta": { "coreVersion": "1.0.0" } }
```

Ошибка возвращается в формате RFC 7807 с путём к полю и понятным сообщением:

```
{ "type": "https://bioauth-optimizer/errors/validation-error",
  "title": "Некорректные входные данные", "status": 400,
  "errors": [{ "path": "model.genuine.mean",
               "message": "должно быть больше impostor.mean" }] }
```

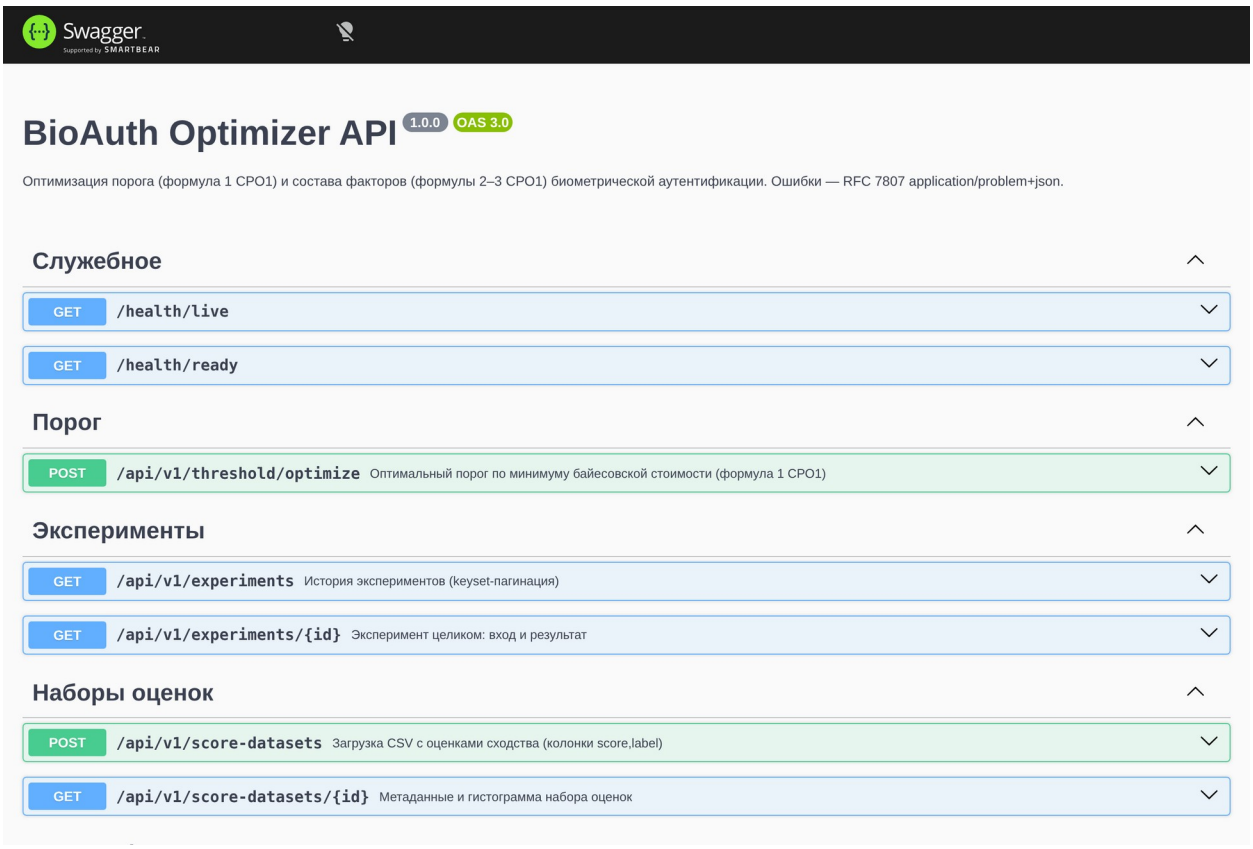


Рисунок 8 — Документация API в Swagger UI

6 РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ

6.1 Требования к оборудованию и программному обеспечению

Таблица 13 — Требования к ЭВМ

Параметр	Минимум	Рекомендуется
Процессор	2 ядра x86-64 или ARM64	4 ядра
Оперативная память	4 ГБ	8 ГБ
Свободное место на диске	3 ГБ (образы Docker)	5 ГБ
Операционная система	Linux, macOS 12+, Windows 10/11 с WSL 2	—
Программное обеспечение	Docker Desktop или Docker Engine с Compose v2	Для разработки: Node.js 22, pnpm 10, GNU Make
Браузер	Chrome, Edge, Firefox, Safari последних версий	—

6.2 Схема развёртывания

Все компоненты запускаются в Docker Compose (рисунок 9). Порядок запуска гарантируется проверками здоровья: db → api (миграции, затем проверка /health/ready) → web. Интерфейс проксирует запросы `/api/*` на сервер внутри сети compose, поэтому браузер обращается к одному адресу и CORS не нужен.

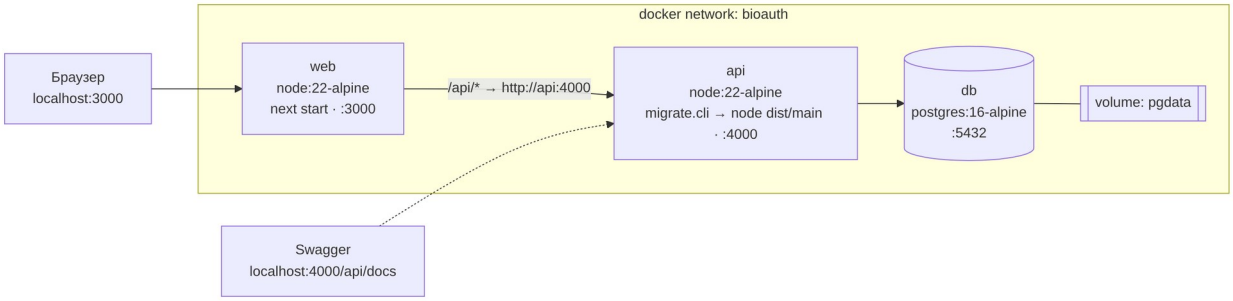


Рисунок 9 — Схема развёртывания

– **Многоэтапная сборка образов.** `turbo prune` оставляет в контексте сборки только нужный сервис и его пакеты; слой с зависимостями кэшируется, пока не меняется lockfile.

– **Образ api** содержит только скомпилированный код, миграции и зависимости для работы. При старте выполняются миграции, затем запускается сервер.
– **Образ web** — автономная (standalone) сборка Next.js.
– Оба процесса работают от непривилегированного пользователя; база данных публикуется только на 127.0.0.1.

6.3 Переменные окружения

Таблица 14 — Переменные окружения

Переменная	Сервис	По умолчанию	Назначение
DATABASE_URL	api	— (обязательна)	Строка подключения к PostgreSQL
PORT	api	4000	Порт HTTP
CORS_ORIGIN	api	http://localhost:3000	Разрешённый источник запросов
MAX_UPLOAD_MB	api	50	Лимит размера CSV
LOG_LEVEL	api	info	Уровень журналирования
API_INTERNAL_URL	web (сборка)	http://api:4000	Адрес сервера для прокси
NEXT_PUBLIC_SWAGGER_URL	web (сборка)	http://localhost:4000/api/docs	Ссылка на Swagger в шапке
POSTGRES_PASSWORD	db, api	bioauth	Пароль БД для локального запуска

Переменные сервера проверяются zod-схемой при старте: без DATABASE_URL процесс завершается с понятной ошибкой.

6.4 Команды управления

Команды собраны в Makefile; `make help` выводит их список с описаниями.

Таблица 15 — Основные команды

Команда	Действие
make up	Собрать и запустить db, api, web; дождаться готовности
make health	Проверить сервер, БД, интерфейс и прокси
make logs SERVICE=api	Журнал сервиса
make down	Остановить контейнеры (данные сохраняются)
make demo	Загрузить демонстрационный каталог из 10 факторов
make setup / make dev	Установка для разработки / запуск в режиме разработки
make check	Проверки CI: форматирование, линтер, типы, тесты
make smoke	Запуск, проверка здоровья и браузерные тесты
make release-zip	Архив исходного кода для сдачи

7 ТЕСТИРОВАНИЕ И ВЕРИФИКАЦИЯ

Главный риск программы — неверный расчёт, поэтому основная часть тестов сосредоточена на вычислительном ядре. Всего 137 автоматических тестов, все проходят; дополнительно четыре браузерных сценария выполняются против запущенных контейнеров.

7.1 Уровни тестирования

Таблица 16 — Пирамида тестов

Уровень	Инструмент	Что проверяется	Тестов
Золотые и property-тесты ядра	Vitest, fast-check	Числа статьи, эталоны SciPy, инварианты, согласие решателей	103
Контракты	Vitest	zod-схемы и сообщения об ошибках	5
Сервер: модульные и интеграционные	Jest, supertest, PostgreSQL	Сценарии с заглушками портов, разбор CSV, все эндпоинты на реальной БД	17
Интерфейс: модульные	Vitest	Форматирование и разбор чисел	12
Браузерные (e2e)	Playwright	Пример статьи даёт верные числа на экране; история; ошибки ввода	4 сценария
Развёртывание	GitHub Actions	docker compose up → проверки здоровья → e2e против контейнеров	каждый push

7.2 Золотые тесты

– **Порог, пример статьи** — все величины с допусками от 10^{-12} (абсолютный для τ_{EER}) до 10^{-9} (для τ^*).
– **Порог против SciPy** — 50 наборов параметров с эталонами `scipy.stats.norm` и `scipy.optimize`; скрипт генерации `tools/generate-fixtures.py` хранится в репозитории.
– **Хвосты распределений** — FAR при z = 8 и z = 10 (около $6{,}2\cdot10^{-16}$ и $7{,}6\cdot10^{-24}$) совпадает со SciPy с относительной погрешностью ≤ 10^{-12}; тест падает, если вычислять 1 − Ф.

– **Факторы, пример статьи** — 31 подмножество, 6 допустимых, рейтинг и метрики как в статье; при B = 2,0 — 3 допустимых, оптимум F1+F2, отношение потерь 1,7415.
– **Регрессия плавающей точки** — F2+F4 (FAR = 10^{-6}) допустим при $FAR_{max} = 10^{-6}$.
– **Сертификат ЦЛП** — ровно 2 итерации, нижняя граница 0,0689288; при FRR = 0 у всех факторов — не более 3 итераций.
– **Перебор бюджета** — точки смены оптимума 0,9; 1,6; 2,1.

7.3 Property-based тесты

Библиотека fast-check генерирует случайные задачи и проверяет свойства, которые должны выполняться всегда (таблица 17). Найденные ею контрпримеры (вырожденный корень в дальнем хвосте, нулевые FRR, «ничьи») превращены в регрессионные тесты.

Таблица 17 — Проверяемые свойства

Свойство	Формулировка
Согласие решателей	Для случайной задачи с $n \le 12$ полный перебор и ЦЛП дают одинаковое E (отн. 10^{-12}) и одинаково определяют допустимость
Оптимальность перебора	Ни один допустимый набор из рейтинга не имеет E меньше оптимума
Монотонность по бюджету	$E^*(B_1) \ge E^*(B_2)$ при $B_1 \le B_2$
Стационарность порога	В невырожденном τ^*: $\lvert E'(\tau^*)\rvert < 10^{-9}$ и $E(\tau^* \pm 10^{-4}) \ge E(\tau^*)$
Аналитика против численной проверки	$\lvert\tau_{аналит} - \tau_{числ}\rvert < 10^{-7}$ для случайных параметров
Точка равных ошибок	$FAR(\tau_{EER}) = FRR(\tau_{EER})$ с отн. погрешностью 10^{-12}
Эмпирический режим в пределе	На $2 \cdot 10^5$ выборках из $N(\mu, \sigma)$ эмпирический τ^* близок к аналитическому ($\le 0{,}02$), потери выше не более чем на 1 %
Точность эмпирического перебора	Перебор всех порогов «в лоб» $O(n^2)$ даёт тот же E^*, что и алгоритм с двумя указателями
Детерминизм	Повторный запуск даёт побайтно одинаковый результат

7.4 Непрерывная интеграция и качество кода

При каждом push и pull request GitHub Actions выполняет: проверку сообщений коммитов (commitlint), форматирования (Prettier), сборку, линтер

(ESLint в строгом режиме с проверкой типов, архитектурными границами и правилами FSD), проверку типов TypeScript и все тесты на сервисе PostgreSQL. Второе задание собирает образы, запускает `docker compose up --wait`, проверяет здоровье и выполняет браузерные тесты против контейнеров. Локально те же проверки запускает `make check`, а git-хуки husky не дают закоммитить неотформатированный код.

7.5 Результаты на примере статьи СРО1

Программа воспроизводит модельный пример раздела 4 статьи (таблица 18).

Таблица 18 — Сравнение результатов программы и статьи

Величина	Статья СРО1	Программа
τ_{EER}	0,531	0,531
FAR = FRR в точке EER	≈ 1,77 %	1,786 %
$E(\tau_{EER})$	0,195	0,196509
τ^*	0,582	0,5818
$FAR(\tau^*)$ / $FRR(\tau^*)$	0,52 % / 6,2 %	0,521 % / 6,230 %
$E(\tau^*)$	0,114	0,114387
Снижение потерь Δ	≈ 41 %	41,8 %
Допустимых наборов при B = 2,5	6 из 31	6 из 31
Оптимальный набор	F2 + F5	F2 + F5, E = 0,019925
Оптимум при B = 2,0	F1 + F2	F1 + F2, E = 0,034700
Рост потерь при B: 2,5 → 2,0	≈ 1,74 раза	1,74 раза

Расхождение в третьем знаке для EER объясняется округлением в статье. Точный расчёт при N(0,72; 0,09) и N(0,30; 0,11) даёт FAR = FRR = 1,786 % и $E(\tau_{EER})$ = 0,1965; τ_{EER} = 0,531 получается точно — это ровно 2,1σ от обоих средних. Отсюда снижение 41,8 %, а не 41 %. Вывод статьи от этого не меняется. Значения программы подтверждены независимым расчётом в SciPy.

8 СВЯЗЬ С МЕТОДАМИ СРО1

Программа реализует ровно формулы (1)–(3) статьи СРО1 и её рекомендации. Таблица 19 показывает, где каждое положение статьи реализовано и чем проверено.

Таблица 19 — Соответствие положений статьи, кода и тестов

Положение статьи СРО1	Модуль кода	Проверка
Формула (1): выбор порога по минимуму $E(\tau) = c_{FA}\cdot FAR + c_{FR}\cdot FRR$	`threshold/bayes-cost.ts`, `optimizers/gaussian-analytic.optimizer.ts`	threshold.golden.test.ts — пример статьи, 50 эталонов SciPy
Сравнение с порогом EER и снижение потерь (41,8 %)	`gaussian-analytic.optimizer.ts` (τ_{EER} в замкнутом виде)	threshold.golden.test.ts; e2e: «Δ = 41,8 %» на экране
Рабочая точка по эмпирическим оценкам	`optimizers/empirical-sweep.optimizer.ts`	empirical.test.ts — перебор «в лоб», предел к нормальной модели
Формула (2): правило «И» для FAR и FRR набора	`fusion/and-fusion.rule.ts`, `subset-evaluator.ts`	factor-selection.golden.test.ts — метрики 6 наборов
Формула (3): ограничения на задержку, стоимость, FAR, PAD	`subset-evaluator.ts`, `numeric/tolerance.ts`	golden — 6 из 31, регрессия F2+F4
Логарифмирование и сведение к целочисленному программированию	`solvers/certified-ilp.solver.ts`, `highs-milp.adapter.ts`	properties — согласие с полным перебором; сертификат: 2 итерации
Таблица 2 и рис. 2б: оптимум F2+F5	`examples/article-sro1.ts`	golden; e2e: «F2 + F5», «6 из 31»
Цена ужесточения бюджета (рост E в 1,74 раза)	`sensitivity/budget-sweep.ts`	golden — точки 0,9 / 1,6 / 2,1, отношение 1,7415

Программа также уточняет статью в двух местах: использует точную форму $\Lambda(x)$ вместо приближения $\Sigma\ FRR_i x_i$ и доказывает оптимальность найденного набора нижней границей, а не только находит его.

9 ОГРАНИЧЕНИЯ И НАПРАВЛЕНИЯ РАЗВИТИЯ

9.1 Ограничения версии 1.0

Таблица 20 — Ограничения и их следствия

Ограничение	Следствие
Биометрия не собирается: оценки и параметры факторов вводятся вручную	Результат хорош настолько, насколько точны введённые данные
Параметры факторов берутся из литературы или паспорта вендора	Фактические FRR и задержка в организации могут отличаться
Найденный порог не применяется автоматически	Перенос τ* в рабочую систему — отдельный шаг
Правило объединения только «И»	Каскады и объединение на уровне оценок не моделируются
Нет пользователей и ролей	Подходит для лаборатории и учебных задач, а не для промышленной эксплуатации

9.2 Направления развития

Следующая версия замыкает контур «измерить → оптимизировать → применить» (рисунок 10): система сама собирает оценки и исходы попыток входа, калибрует порог и состав факторов методами CPO1 и после утверждения администратором применяет новую политику входа. Методы статьи при этом не меняются — меняется только источник чисел. Техническое задание — `docs/10-live-biometrics-spec.md`, задачи — в трекере GitHub (вехи v2.1 и v2.2).

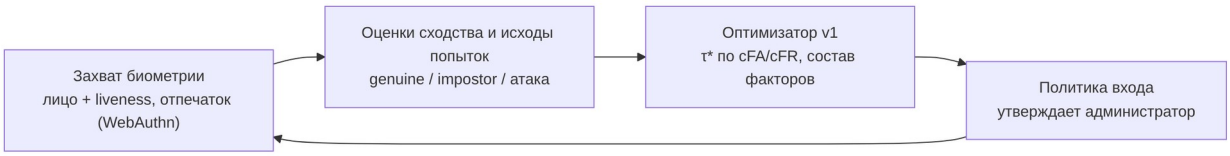


Рисунок 10 — Замкнутый контур калибровки (версия 2)

– сервис распознавания лиц (YuNet, SFace, MiniFASNet в ONNX Runtime) с контролем качества кадра и проверкой «живости» по трём случайным командам;
– вход по passkey (WebAuthn) с проверкой отпечатка на устройстве пользователя;

– шифрование биометрических шаблонов AES-256-GCM, согласие субъекта и удаление по запросу;
– калибровка по накопленным оценкам, версионируемые политики входа, испытание защиты от атак на предъявление (APCER/BPCER по ISO/IEC 30107-3);
– метаэвристики (PSO, GA, NSGA-II) для объединения трёх и более модальностей и многокритериального фронта Парето.

ЗАКЛЮЧЕНИЕ

Разработано программное средство BioAuth Optimizer, которое реализует методы статьи CPO1 полностью и с доказуемой точностью: оптимальный порог по минимуму байесовской стоимости в аналитическом и эмпирическом режимах, оптимальный состав факторов двумя независимыми решателями, анализ чувствительности к бюджету.

Точность подтверждена 137 автоматическими тестами: совпадением с независимым расчётом в SciPy до 10^{-12}, согласием двух решателей на случайных задачах, регрессионными тестами граничных случаев плавающей точки. Модельный пример статьи воспроизводится полностью; уточнённые значения (снижение потерь на 41,8 %) объяснены.

Архитектура — модульный монолит с чистым вычислительным ядром, сервером по Clean Architecture и интерфейсом по Feature-Sliced Design — соблюдает принципы SOLID и допускает расширение без переделки: в версии 2 ядро используется без изменений для калибровки по собственным биометрическим данным. Программа запускается одной командой в Docker и проверяется в CI при каждом изменении.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1) Мекенбаев Д. К. Многокритериальная оптимизация параметров систем биометрической аутентификации // СРО1 по дисциплине «Методы и инструменты оптимизации». — Алматы: Университет «Туран», 2026.
2) ISO/IEC 19795-1:2021. Information technology — Biometric performance testing and reporting — Part 1: Principles and framework.
3) ISO/IEC 30107-3:2023. Information technology — Biometric presentation attack detection — Part 3: Testing and reporting.
4) Huangfu Q., Hall J. A. J. Parallelizing the dual revised simplex method // Mathematical Programming Computation. — 2018. — Vol. 10. — P. 119–142. (решатель HiGHS)
5) Jain A. K., Ross A., Nandakumar K. Introduction to Biometrics. — New York: Springer, 2011. — 311 p.
6) Miettinen K. Nonlinear Multiobjective Optimization. — Boston: Kluwer, 1999. — 298 p. (метод ε-ограничений)
7) Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. — Prentice Hall, 2017. — 432 p.
8) Feature-Sliced Design — архитектурная методология для фронтенд-проектов [Электронный ресурс]. — URL: https://feature-sliced.design
9) Nottingham M., Wilde E. RFC 7807. Problem Details for HTTP APIs. — IETF, 2016.
10) Virtanen P. et al. SciPy 1.0: fundamental algorithms for scientific computing in Python // Nature Methods. — 2020. — Vol. 17. — P. 261–272.

ПРИЛОЖЕНИЕ А. Исходные данные и результаты примера СРО1

Таблица 21 — Каталог факторов (таблица 2 статьи СРО1)

Код	Фактор	FAR	FRR	Задержка, мс	Стоимость	PAD
F1	Лицо (ArcFace) + PAD	$1{,}0\cdot10^{-4}$	0,020	350	1,0	да
F2	Отпечаток пальца	$5{,}0\cdot10^{-5}$	0,015	250	0,6	да
F3	Голос	$5{,}0\cdot10^{-3}$	0,060	900	0,4	нет
F4	Поведенческая биометрия	$2{,}0\cdot10^{-2}$	0,050	60	0,3	нет
F5	Аутентификатор FIDO2	$1{,}0\cdot10^{-6}$	0,005	400	1,5	нет

Ограничения: L_{max} = 1000 мс, B = 2,5, $FAR_{max} = 10^{-6}$, не менее одного фактора с PAD; c_{FA} = 10, c_{FR} = 1. Признак PAD в статье явно не указан и восстановлен по рисунку 2б: допустимы F2+F4 и F1+F2+F4, но не F4+F5 и не F5 отдельно, следовательно PAD имеют F1 и F2.

Таблица 22 — Допустимые наборы при B = 2,5 (результат программы)

Ранг	Набор	FAR	FRR	L, мс	C	E
1	F2+F5	$5{,}0\cdot10^{-11}$	0,019925	650	2,1	0,019925
2	F1+F5	$1{,}0\cdot10^{-10}$	0,024900	750	2,5	0,024900
3	F1+F2	$5{,}0\cdot10^{-9}$	0,034700	600	1,6	0,034700
4	F2+F4	$1{,}0\cdot10^{-6}$	0,064250	310	0,9	0,064260
5	F2+F4+F5	$1{,}0\cdot10^{-12}$	0,068929	710	2,4	0,068929
6	F1+F2+F4	$1{,}0\cdot10^{-10}$	0,082965	660	1,9	0,082965

Таблица 23 — Эталонные значения расчёта порога ($\mu_G = 0{,}72$, $\sigma_G = 0{,}09$, $\mu_I = 0{,}30$, $\sigma_I = 0{,}11$, $c_{FA} = 10$, $c_{FR} = 1$)

Величина	Значение	Допуск теста
τ_{EER}	0,531	10^{-12}
$FAR(\tau_{EER}) = FRR(\tau_{EER})$	0,017864420562817	отн. 10^{-12}
$E(\tau_{EER})$	0,196508626190982	отн. 10^{-12}
τ^*	0,581785716220834	10^{-9}
$FAR(\tau^*)$	0,005208237451590	отн. 10^{-9}
$FRR(\tau^*)$	0,062304237539322	отн. 10^{-9}
$E(\tau^*)$	0,114386612055219	отн. 10^{-12}
Δ	0,41790539 (41,8 %)	10^{-8}

ПРИЛОЖЕНИЕ Б. Структура репозитория

```
bioauth-optimizer/
├── apps/
│   ├── api/                    сервер NestJS
│   │   ├── migrations/         SQL-миграции
│   │   ├── src/modules/        threshold, factor-selection, factor-catalogs,
│   │   │                       score-datasets, experiments, examples
│   │   ├── src/shared/         config, database, http, logging, health, swagger
│   │   └── test/               unit и e2e (Jest + supertest)
│   └── web/                    интерфейс Next.js
│       ├── app/                маршруты
│       ├── src/                app, pages, widgets, features, entities, shared (FSD)
│       └── e2e/                браузерные тесты Playwright
├── packages/
│   ├── optimization-core/      вычислительное ядро (src/, test/)
│   ├── contracts/              zod-схемы API
│   └── config/                 общие настройки TypeScript
├── docker/                     Dockerfile сервера и интерфейса
├── docs/                       документация, ADR, диаграммы
├── examples/                   демонстрационные данные (CSV, каталоги JSON)
├── tools/                      генерация эталонов SciPy
├── .github/workflows/ci.yml    непрерывная интеграция
├── docker-compose.yml
└── Makefile
```